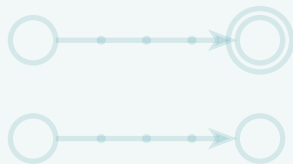


Лекция

30–31 марта

Минимизация автоматов



Различимость состояний

Что значит имя? Роза пахнет розой, хоть розой назови её, хоть нет.

— Уильям Шекспир

Зачем минимизировать?

Таблица переходов — память автомата в железе и в коде.

Лишние состояния — лишние строки таблицы, лишние проверки, лишние регистры.

Пример

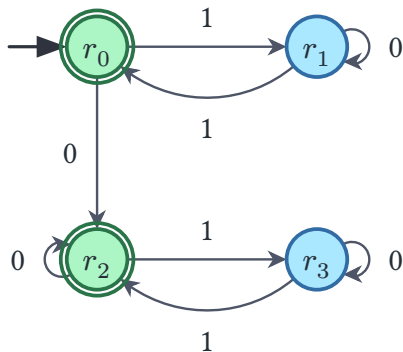
Один и тот же язык «чётное число единиц» распознают автомат из двух состояний и автомат из четырёх.

Второй работает, но половина его состояний — балласт.

Нужно найти лишние состояния и доказать, что после чистки их не осталось.

Ответы — алгоритм таблицы различимостей и теорема о единственности минимального автомата.

Четыре состояния вместо двух



Пример

Похожи пары r_0, r_2 и r_1, r_3 : по 0 обе остаются в своей паре, по 1 обе переходят в соседнюю.

Проверить совпадение ответов перебором слов нельзя — нужен алгоритм.

Эквивалентность состояний

Определение 1

Состояния p, q **эквивалентны**, обозначается $p \sim q$, если на любом слове w оба принимают или оба отвергают:

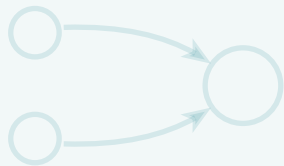
$$\hat{\delta}(p, w) \in F \Leftrightarrow \hat{\delta}(q, w) \in F.$$

Иначе состояния **различимы**: есть слово-свидетель w , разводящее их ответы.

Пример

Слово 1 различает r_0 и r_1 : из r_0 оно отвергается, из r_1 принимается. Слово 10 тоже: путь $r_0 \rightarrow r_1 \rightarrow r_1$ отвергается, путь $r_1 \rightarrow r_0 \rightarrow r_2$ принимается.

Склейка эквивалентных состояний не меняет язык — их склейка и есть минимизация.



Минимизация

Совершенство достигается не тогда, когда уже нечего прибавить, но когда уже ничего нельзя отнять.

— Антуан де Сент-Экзюпери

Таблица различимостей

Клетка таблицы — пара состояний, пометка — доказанная различимость, пустая клетка — кандидат на склейку.

1. Завести клетки для всех неупорядоченных пар $p \neq q$.
2. Пометить пары, где ровно одно состояние принимающее: их разводит пустое слово ε .
3. Итерация: если пара $(\delta(p, a), \delta(q, a))$ уже помечена, пометить и (p, q) .
4. Повторять, пока таблица меняется.
5. Непомеченные пары — эквивалентные состояния: склеить их.

Различимость распространяется назад по дугам: если w разводит образы, то aw разводит предшественников.

На остановке пустая клетка означает эквивалентность — любой свидетель конечен и был бы найден очередным проходом.

Каждая итерация просматривает все клетки, поэтому время работы — $O(n^2 \cdot |\Sigma|)$.

Первый шаг разметки

Прогоним алгоритм на автомате $r_0, \dots, r_3$ и выпишем его таблицу переходов.

Состояние	принимающее	по 0	по 1
r_0	да	r_2	r_1
r_1	нет	r_1	r_0
r_2	да	r_2	r_3
r_3	нет	r_3	r_2

Пустое слово разводит пары с ровно одним принимающим состоянием: (r_0, r_1) , (r_0, r_3) , (r_1, r_2) , (r_2, r_3) . Кандидаты на склейку — (r_0, r_2) и (r_1, r_3) .

Нулевой шаг не пустой

Четыре пары из списка помечаются до всякой итерации — продолжение ε уже даёт разные ответы, а пропуск шага склеил бы принимающее состояние с отвергающим.

Итерации разметки

Пара	по 0	по 1	шаг 0	итог
(r_0, r_1)	(r_2, r_1)	(r_1, r_0)	помечена	различимы
(r_0, r_2)	(r_2, r_2)	(r_1, r_3)	—	пусто
(r_0, r_3)	(r_2, r_3)	(r_1, r_2)	помечена	различимы
(r_1, r_2)	(r_1, r_2)	(r_0, r_3)	помечена	различимы
(r_1, r_3)	(r_1, r_3)	(r_0, r_2)	—	пусто
(r_2, r_3)	(r_2, r_3)	(r_3, r_2)	помечена	различимы

Образы кандидатов — пары (r_1, r_3) и (r_0, r_2) — не помечены, и итерация ничего не добавляет: таблица остановилась.

Вырожденные образы

Клетки для пар вида (r_2, r_2) в таблице нет: состояние с самим собой разводить нечем.

Склейка состояний

Таблица дала $r_0 \sim r_2$ и $r_1 \sim r_3$.

Классы эквивалентности: $s_0 = \{r_0, r_2\}$ — принимающий блок с началом, $s_1 = \{r_1, r_3\}$ — отвергающий.

Класс	по 0	по 1
s_0	s_0	s_1
s_1	s_1	s_0

По 0 класс остаётся в себе, по 1 уходит в соседний — переходы определены.

Пример

Слово 0110 в новом автомате ведёт $s_0 \rightarrow s_0 \rightarrow s_1 \rightarrow s_0 \rightarrow s_0$ — принято.

В старом: $r_0 \rightarrow r_2 \rightarrow r_3 \rightarrow r_2 \rightarrow r_2$ — тоже принято, получился тот самый автомат чётности.

Единственность минимального ДКА

Алгоритм гарантирует автомат без эквивалентных пар.

Остался вопрос: единственный ли ответ, или другой порядок работы даст другой автомат?

Теорема 1 (Единственность минимального ДКА)

Каждый регулярный язык имеет единственный минимальный ДКА — с точностью до изоморфизма, то есть переименования состояний.

Теорема 2 (Лемма о минимальности)

В минимальном ДКА любые два состояния различимы, а каждое достижимо.

Неразличимая пара склеилась бы, недостижимое состояние удалилось бы — автомат вышел бы меньше для того же языка.

Доказательство: слова строят изоморфизм

Доказательство

Докажем построением изоморфизма: свяжем состояния, в которые ведёт одно и то же слово.

Определённость. Если x и y ведут в M_1 в одно состояние, суффиксы дают одинаковый ответ, и $x \simeq_L y$. Разные состояния M_2 лемма разводит свидетелем — слова оказываются различимы, противоречие.

Инъективность. Разные состояния M_1 лемма разводит свидетелем z : $xz \in L$, $yz \notin L$, и образы совпасть не могут.

Сюръективность. Каждое состояние минимального автомата достижимо некоторым словом.

Начальные состояния отвечают ε , принимающие — словам из L , переход по a переводит состояние x в состояние xa — это изоморфизм.

Каноническая форма

Минимальный ДКА — портрет языка, а не удача конструктора.

Два автомата распознают один язык тогда и только тогда, когда их минимальные формы изоморфны.

Проверка эквивалентности — конечная процедура: минимизировать оба и сравнить.

Пример

Два выражения для языка слов, кончающихся на a b : $(a \mid b)^*ab$ и $b^*aa^*b(b^*aa^*b)^*$.

Записи разные, а минимальный автомат у них один и тот же — сравнение канонических форм и решает вопрос об эквивалентности.

Число состояний минимального ДКА — объективная мера сложности языка.



Классы неразличимости

Напрасно пытаться посредством большего делать то, что может быть сделано посредством меньшего.

— Уильям Оккам

Неразличимость по продолжению

Определение 2

Для языка L слова x, y **неразличимы по продолжению**, $x \simeq_L y$, если

$$\forall z \in \Sigma^*. xz \in L \Leftrightarrow yz \in L.$$

Иначе слова различимы: продолжение z разводит их ответы.

Пример

Для языка слов, заканчивающихся на 01, продолжение 1 разводит ε и 0: $1 \notin L$, а $01 \in L$.

Отношение $\simeq_L$ — эквивалентность: рефлексивность, симметричность и транзитивность наследуются от равенства ответов.

Это та же конструкция классов эквивалентности и фактор-множеств.

Остаточные языки

Классам $\simeq_L$ удобно дать имя по тому, что слово оставляет на будущее.

Определение 3

Остаточный язык слова x относительно L :

$$C_x = \{z \in \Sigma^* \mid xz \in L\}.$$

Классы и остаточные языки — одна и та же информация:

$$x \simeq_L y \iff C_x = C_y.$$

Остаточный язык — всё, что язык собирается спросить о слове в будущем.

Состояние автомата хранит в точности его — и ничего больше.

Три класса для языка «01»

Для языка слов, заканчивающихся на 01, остаточных языков ровно три.

Класс слов	Остаточный язык
кончающиеся на 01	ε и все слова, кончающиеся на 01
кончающиеся на 0	слово 1 и все слова, кончающиеся на 01
остальные слова	все слова, кончающиеся на 01

Пример

Пустое слово и 1 лежат в одном классе: оба дают язык продолжений, кончающихся на 01.

Слово 0 уже допускает продолжение 1: выходит $01 \in L$, поэтому его класс другой.

Три класса — три состояния q_0, q_1, q_2 автомата для этого языка: слова одного класса ведут в одно состояние.

Теорема Майхилла–Нерода

Теорема 3 (Майхилл — Нерод, 1958)

Для языка L следующие утверждения эквивалентны:

1. L регулярен.
2. Отношение $\simeq_L$ имеет конечный индекс.
3. Язык L — объединение классов правой конгруэнции конечного индекса (эквивалентности, согласованной с дописыванием символа справа).

Состояниями минимального автомата служат классы $\simeq_L$.

Индекс отношения — точное число состояний минимального ДКА.

Конечный индекс даёт регулярность и готовый автомат, бесконечный — доказательство нерегулярности.

Доказательство: цикл импликаций

Доказательство

Докажем эквивалентность трёх утверждений по циклу.

Регулярность $\Rightarrow$ конечный индекс. Если два слова ведут ДКА в одно состояние, любой суффикс даёт для обоих одинаковый ответ, и слова неразличимы. Каждое состояние собирает целый класс $\simeq_L$, и классов не больше $|Q|$ — индекс конечен.

Конечный индекс $\Rightarrow$ конгруэнция. Из $x \simeq_L y$ следует $xa \simeq_L ya$: продолжение z для пары xa, ya — то же слово az для пары x, y . Язык L — объединение классов, пересекающих L .

Конгруэнция $\Rightarrow$ регулярность. Фактор-автомат: состояния — классы, начальное — $[\varepsilon]$, переход по a ведёт из $[x]$ в $[xa]$ независимо от представителя.

Принимающие — классы со словами из L , и автомат распознаёт ровно L .

Нерегулярность через классы

Критерий работает и в отрицательную сторону.

Пример

Для $L = \{a^n b^n\}$ рассмотрим слова $a, a^2, a^3, \dots$

Для любых $i < j$ продолжение $z = b^i$ разводит их:

$$a^i b^i \in L, \quad a^j b^i \notin L.$$

Все степени a попарно различимы — классов бесконечно много.

Бесконечный индекс $\simeq_L$ — и язык $a^n b^n$ не регулярен.

В отличие от накачки, доказательство конструктивно: предъявлено бесконечное множество попарно различных слов.

Два инструмента рядом

Лемма о накачке и теорема Майхилла–Нерода отвечают на один вопрос — регулярен ли язык — инструментами разной силы.

Инструмент	Что даёт	Граница
Лемма о накачке	опровержение регулярности	необходимое условие, не достаточное
Теорема Майхилла–Нерода	критерий в обе стороны и минимальный автомат	нужно уметь считать классы

Число состояний минимального ДКА равно числу классов $\simeq_L$: мера сложности языка — в самом языке, а не в руках конструктора.

* Разбиения вместо пар

Эквивалентность можно искать и разбиением состояний на блоки: начальное — принимающие и остальные, дальше блоки расщепляются.

1. Блок расщепляется по символу a , если его состояния ведут в разные блоки.
2. Расщепления повторяются, пока разбиение меняется.
3. Стабильное разбиение — те же классы, что дают пустые клетки таблицы.

Алгоритм	Идея	Сложность
Мур, таблица различимостей	пометка различных пар и склейка неразличимых	$O(n^2 \cdot \Sigma)$
Мур, уточнение разбиения	блок расщепляется, когда состояния расходятся по блокам	$O(n^2 \cdot \Sigma)$
Хопкрофт (1971)	при расщеплении перестраивается только меньшая половина	$O(n \cdot \Sigma \cdot \log n)$

* Прогон Хопкрофта

Автомат с состояниями $0, 1, 2, 3, 4$ и принимающими $\{3, 4\}$: по a переходы $0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 3, 4 \rightarrow 4$, по b все состояния уходят в 4 .

Начальное разбиение: $\{0, 1, 2\}, \{3, 4\}$.

По a блок $\{0, 1, 2\}$ расщепляется: состояния 0 и 1 переходят внутрь блока, состояние 2 — в принимающий.

Разбиение стало $\{0, 1\}, \{2\}, \{3, 4\}$, и блок $\{0, 1\}$ расщепляется тем же символом: 0 переходит в 1 (тот же блок), 1 — в 2 (другой).

По b всё устойчиво: каждое состояние переходит в 4 .

Итог — $\{0\}, \{1\}, \{2\}, \{3, 4\}$, четыре состояния вместо пяти.

Что мы поняли: минимизация

От похожих пар состояний — к канонической форме языка: склейка эквивалентных состояний, таблица различимостей и теорема о классах $\simeq_L$.

- Состояния эквивалентны, если их не различает ни одно слово — склейка даёт минимизацию.
- Таблица различимостей находит все различимые пары за $O(n^2 \cdot |\Sigma|)$.
- Теорема Майхилла–Нерода: классы $\simeq_L$ — состояния минимального ДКА.
- Минимальный ДКА единствен с точностью до изоморфизма — каноническая форма языка.
- Конечность модели делает пустоту, эквивалентность и минимизацию разрешимыми.

Вопросы для самопроверки

Вопросы для самопроверки — о минимизации

1. Почему в минимальном ДКА любые два состояния различимы? Что сломалось бы иначе?
2. Найдите слово-свидетель для пары (r_0, r_1) автомата с четырьмя состояниями и объясните, почему для пары (r_0, r_2) свидетеля нет.
3. Опишите классы $\simeq_L$ для языка слов с чётным числом единиц. Сколько их и как выглядят свидетели между классами?
4. Докажите через Майхилла–Нерода нерегулярность языка палиндромов над $\{a, b\}$. Сколько попарно различных слов достаточно предъявить?

Что почитать

- m23 — «Минимизация ДКА», «Теорема Майхилла–Нерода», «Минимизация ДКА (алгоритм Хопкрофта)».